

SURROGATE CONVOLUTIONAL NEURAL NETWORK MODELS FOR STEADY COMPUTATIONAL FLUID DYNAMICS SIMULATIONS*

MATTHIAS EICHINGER[†], ALEXANDER HEINLEIN[‡], AND AXEL KLAWONN[§]

Abstract. A convolution neural network (CNN)-based approach for the construction of reduced order surrogate models for computational fluid dynamics (CFD) simulations is introduced; it is inspired by the approach of Guo, Li, and Iori [X. Guo, W. Li, and F. Iorio, Convolutional neural networks for steady flow approximation, in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, New York, USA, 2016, ACM, pp. 481–490]. In particular, the neural networks are trained in order to predict images of the flow field in a channel with varying obstacle based on an image of the geometry of the channel. A classical CNN with bottleneck structure and a U-Net are compared while varying the input format, the number of decoder paths, as well as the loss function used to train the networks. This approach yields very low prediction errors, in particular, when using the U-Net architecture. Furthermore, the models are also able to generalize to unseen geometries of the same type. A transfer learning approach enables the model to be trained to a new type of geometries with very low training cost. Finally, based on this transfer learning approach, a sequential learning strategy is introduced, which significantly reduces the amount of necessary training data.

Key words. Convolutional neural networks, computational fluid dynamics, reduced order surrogate models, U-Net, transfer learning, sequential learning

AMS subject classifications. 35Q30, 68T07, 68T10, 65N22

1. Introduction. The development of machine learning techniques for the solution of differential equations is an important topic in the new, rapidly evolving field of scientific machine learning (SciML) [3]. For instance, machine learning techniques may be used in order to discretize the differential equations, in order to enhance classical numerical models and methods, or as reduced order surrogate models.

Examples of methods, where neural networks are used as discretizations for partial differential equations are the approaches by Lagaris et al. [24], physics-informed neural networks (PINNs) by Raissi et al. [31], and the Deep Ritz approach by E and Yu [9]. In [15], PINNs have recently been applied to a parameter identification problem for systems of ordinary differential equations.

There is also a large number of recent publications on hybrid algorithms using machine learning techniques to enhance classical numerical methods for solving differential equations; see, e.g., [4, 17, 18, 19, 33].

Here, we are interested in constructing surrogate models for computational fluid dynamics (CFD) simulations. This is particularly important since CFD simulations arise in many application areas, such as civil and mechanical engineering, meteorology, geosciences, or medical science. Accordingly, there is a wide range of settings and fluids with varying complexity in their modeling. In most cases, however, the computational work in order to obtain accurate results is considerably high. Therefore, the development of reduced order models, which may reduce the computational cost, is of great interest. There is a wide range of classical model order reduction (MOR) techniques, such as principal component analysis

*Received December 12, 2020. Accepted October 06, 2021. Published online on March 18, 2022. Recommended by Xiao-Chuan Cai.

[†]Department of Mathematics and Computer Science, University of Cologne, Weyertal 86–90, 50931 Köln, Germany (eichingm@mail.uni-koeln.de, axel.klawonn@uni-koeln.de).
<http://www.numerik.uni-koeln.de>.

[‡]Delft Institute of Applied Mathematics, Delft University of Technology, The Netherlands (a.heinlein@tudelft.nl).

[§]Center for Data and Simulation Science, University of Cologne, Germany, <http://cds.uni-koeln.de>.

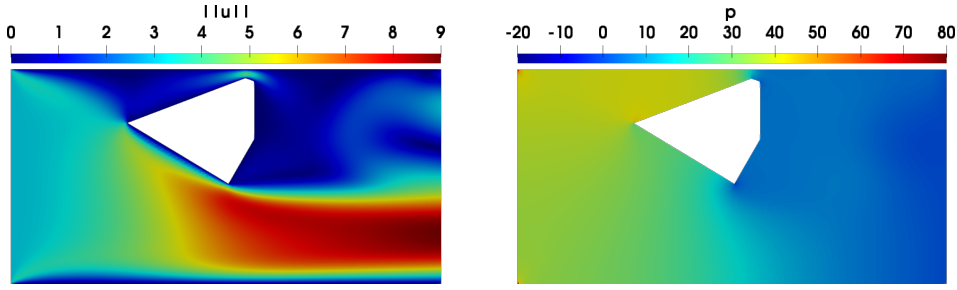


FIG. 2.1.1. Solution of the steady Navier-Stokes equations (2.1) with boundary conditions (2.2) corresponding to the configuration in Figure 2.2.

(PCA) or proper orthogonal decomposition (POD), Reduced Basis (RB), or simplified physics methods. In this regards, we refer to the extensive literature, e.g., [29, 30, 32, 36].

Here, we are interested in the use of artificial neural networks as reduced order surrogate models for CFD simulations. One approach of this category is the hybrid algorithm introduced in [8], which uses classical reduced basis solvers as the last layer in a neural network, to approximate the solution of parametrized PDEs. Also in [11], the authors build a reduced order model for cardiac electrophysiology by combining a convolutional auto-encoder and a deep feedforward neural network. In contrast to most other approaches, we are interested in geometry-dependent flow predictions, where the geometry is used as the input for the neural network and the whole flow field is obtained as the output. Our approach is inspired by the work of Guo, Li, and Iorio [16] and has already been partly presented in [10]. As in the original work [16], we employ convolutional neural networks (CNNs) in order to map from an image of the geometry to images of the resulting flow field, making use of the strengths of CNNs in processing image data. While, in [16], the authors employ a plain bottleneck CNN to minimize a mean squared error as the loss function, we extended the approach by using U-Net type network architectures, which have been introduced in Ronneberger et al. [34] for biomedical image segmentation, as well as more sophisticated choices for the loss function; this includes a spatial weighting approach for the loss function in order to improve the predictions near the obstacle wall. In addition to an extensive optimization of further hyper parameters, we introduce approaches to improve the generalization with respect to new types of geometries while reducing the amount of necessary training data.

In order to generate synthetic flow data for the training and validation phase, we use a software pipeline including the mesh generation tool snappyHexMesh and the simpleFoam CFD solver; both are part of OpenFOAM 5.0 [14]. In order to implement and train the neural networks, we use Keras 2.2.4 [6] with Tensorflow 1.12 [2] backend.

This paper is structured as follows. First, in Section 2, we introduce the considered stationary CFD boundary value problem. Then, we describe our approach of constructing a surrogate CNN model and the employed network architectures in Section 3 and the data generation process in Section 4. In Sections 5 and 6, we describe the training procedure and implementation and efficiency of the neural networks, respectively. Next, we provide results for two types of obstacle geometries in Section 7 and discuss a transfer learning approach for extending the model to another type of geometries in Section 8. Finally, before we conclude the paper in Section 10, we introduce an iterative sequential learning approach, which enables us to significantly reduce the amount of necessary training data in Section 9.

2. Stationary flow problem. As the model problem, we consider the stationary flow of an incompressible Newtonian fluid with kinematic viscosity $\nu > 0$ within a computational

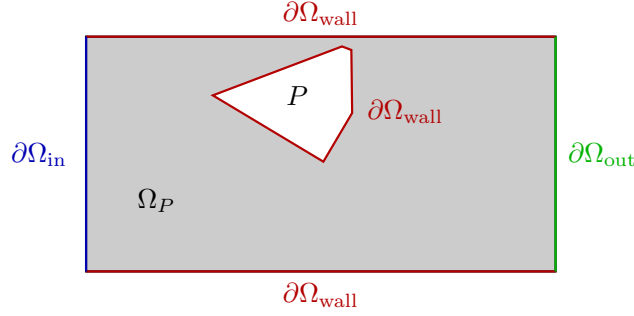


FIG. 2.2. The computational domain Ω_P is a channel Ω of length 6 and width 3 with a polygonal obstacle P . We prescribe an inflow velocity at $\partial\Omega_{\text{in}}$, no-slip boundary conditions on $\partial\Omega_{\text{wall}}$, and do-nothing boundary conditions on $\partial\Omega_{\text{out}}$.

domain Ω_P . This can be described by the steady Navier–Stokes equations

$$(2.1) \quad \begin{aligned} -\nu\Delta u + (u \cdot \nabla) u + \nabla p &= f & \text{in } \Omega_P, \\ \nabla \cdot u &= 0 & \text{in } \Omega_P, \end{aligned}$$

where u and p are the velocity and pressure variables. We will only consider the case where the volume force f is zero.

As a proof of concept, we will restrict ourselves to geometries of a specific type. In particular, we consider two-dimensional channels $\Omega_P := [0, 6] \times [0, 3] \setminus P$, where $P \subset [0, 6] \times [0, 3]$ is a polygonal star-shaped domain; see Figure 2.1 for an example with corresponding solution. Moreover, we apply the boundary conditions

$$(2.2) \quad \begin{aligned} u &= \begin{pmatrix} 3 \\ 0 \end{pmatrix} & \text{on } \partial\Omega_{\text{in}}, \\ \frac{\partial u}{\partial n} - pn &= 0 & \text{on } \partial\Omega_{\text{out}}, \text{ and} \\ u &= 0 & \text{on } \partial\Omega_{\text{wall}}, \end{aligned}$$

where n is the outward pointing normal vector; cf. Figure 2.2. In particular, we prescribe an inflow velocity on the left boundary of the channel and a do-nothing natural boundary condition at the right boundary of the channel. At the upper and lower boundaries as well as the boundary of the obstacle P we impose no-slip boundary conditions.

3. Surrogate convolutional neural network. Our main idea is to train a convolutional neural network (CNN)

$$\begin{aligned} \mathcal{CNN} : \mathbb{R}^{w \times h} &\rightarrow \mathbb{R}^{2 \times w \times h} \\ g &\mapsto u = \begin{pmatrix} u_x \\ u_y \end{pmatrix} \end{aligned}$$

which maps from an image representation of the geometry g to the corresponding velocity field u consisting of an image representations of its x component u_x and its y component u_y . The velocity field is obtained by solving (2.1) with boundary conditions (2.2). Here, w is the width of the input image and the output images and h is the corresponding height. Our

approach is inspired by the work of Guo, Li, and Iorio [16] and has already been presented partly in [10].

Convolutional neural networks [25] are specialized neural networks for data with a tensor product grid-like topology; see also [13, Chapter 9]. Examples for this type of data are, e.g., one-dimensional time series or 2D and 3D images. Whereas the application of one layer in a dense neural networks can be written as

$$y = \alpha(Wx + b)$$

with input vector x , output vector y , dense weight matrix W , bias vector b , and activation function $\alpha(\cdot)$, in a CNN, the matrix multiplication with the dense matrix W is replaced by a convolutional operation. Moreover, convolutional neural networks typically use so-called pooling operations. Both convolution and pooling operations are specialized operations, which only allow for interaction of neighboring neurons, in the sense of the underlying grid-structure of the data. This is implemented by local multiplications with a smaller filter F matrix, which is moved point by point over the whole data grid. In particular, during the training, the coefficients of the filter matrix F are trained. Then, the application of a convolution or pooling operation can also be written as a matrix vector multiplications with a sparse matrix S , which can be obtained by assembly of the local filter matrix. Hence, the application of a convolutional layer corresponds to

$$y = \alpha(Sx + b).$$

Convolution and pooling operations may reduce the size of data vector. However, one typically uses multiple filters corresponding to the multiplication with multiple sparse matrices S_i . Hence, we also obtain multiple output vectors (channels)

$$y_i = \alpha(S_i x + b) \quad i = 1, \dots, N.$$

This helps to prevent the loss of important features from a radical reduction of the dimension; cf., e.g., [13, Chapter 9] and [5, Chapter 5] for more details on convolutional neural networks.

In Figure 3.1, the different types of layers of the employed convolutional neural network architectures are visualized as boxes with different width and height to account for the dimension of the output image of the corresponding layer as well as the length accounting for the number of channels of the output. We consider three different types of convolutional layers:

Convolution: We apply padding before the convolution, such that the dimension of the data is kept the same.

Down-convolution: We apply striding, i.e., we increase the shift of the local filter within the convolution operation, such that the data dimension is reduced. In this case, we increase the number of channels.

Up-convolution: We use a transposed convolution with striding to increase the dimension of the data. At the same time, we reduce the number of channels.

As activation functions, we use the identity (*linear*) or the Rectified Linear Unit (ReLU) [12, 21, 27] function (*ReLU*). For more details on padding and striding, see also [13, Chapter 9] and [5, Chapter 5].

In contrast to other works, which use dense neural networks to approximate the solutions of fixed boundary value problems itself, such as [9, 31], we are here interested in constructing a functional relation between images of the geometry and the solution. This means that we are dealing with given discretization of the input and output, whereas other approaches use the neural network itself as the discretization of the PDE. Therefore, in contrast to those approaches, we are able to incorporate a variation of the geometry.

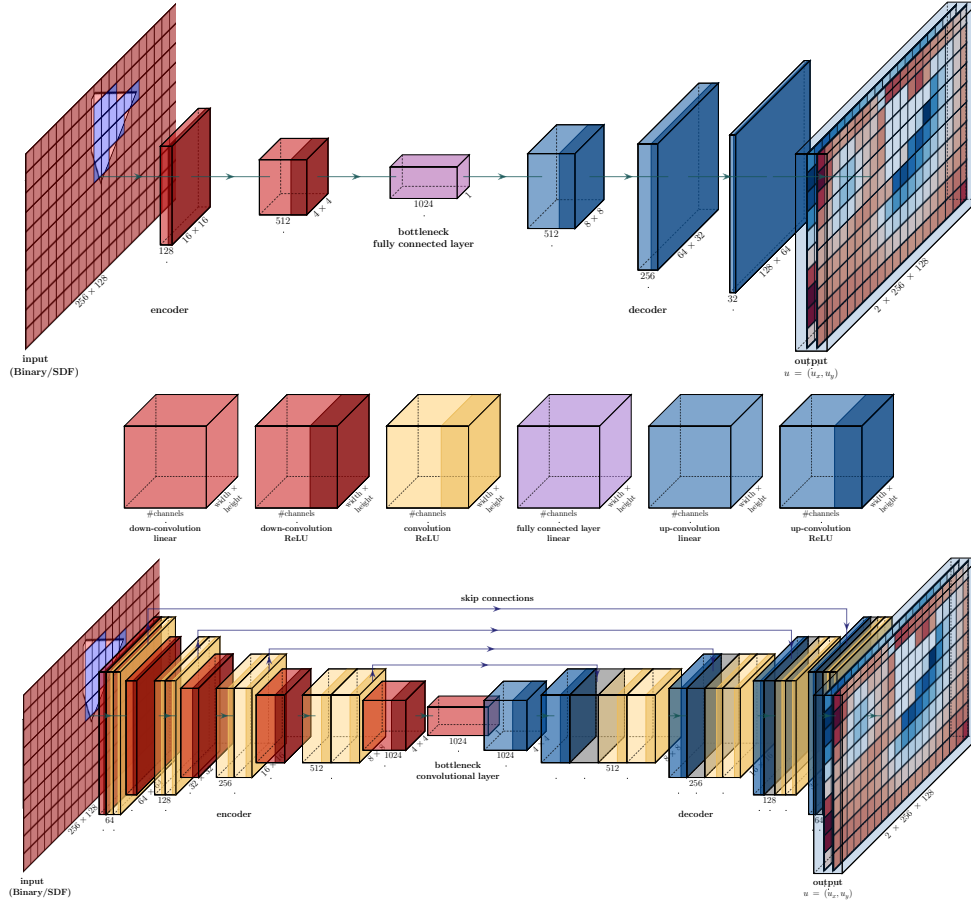


FIG. 3.1. Architectures of the two considered convolutional neural networks: classical bottleneck CNN (top) and U-Net (bottom); cf. Section 3 for more details on the graphical representation. Only one decoder path is depicted; cf. Section 3.2 and Figure 3.2. Visualization using [1].

3.1. Network architectures. As surrogate models, we use **two different convolutional neural network architectures**.

Bottleneck CNN [16]. As the first network architecture, we consider the CNN from the work [16]; see Figure 3.1 (top). **This CNN has a bottleneck structure** such that a reduced dimension latent space of dimension 1 024 is learned from the data. **The first part of the network is an encoder**, the **second part a corresponding decoder**.

U-Net [34]. The second network architecture is inspired by the U-Net introduced by Ronneberger, Fischer, and Brox in [34]; see Figure 3.1 (bottom). **Our network follows the basic structure of the U-Net but the convolutional layers are adapted to our problem**. Similar to the previously described CNN, it is also a convolutional neural network with bottleneck structure. However, **additional skip connections are introduced as a regularization which is necessary due to the high number of convolutional layers**. At the same time, we loose the property that all geometrical information has to be compressed into a vector of length 1 024. Originally, the U-Net has been introduced in the context of biomedical image segmentation, however, as we will observe in Section 7, **it also works very well for the prediction of flow fields**.

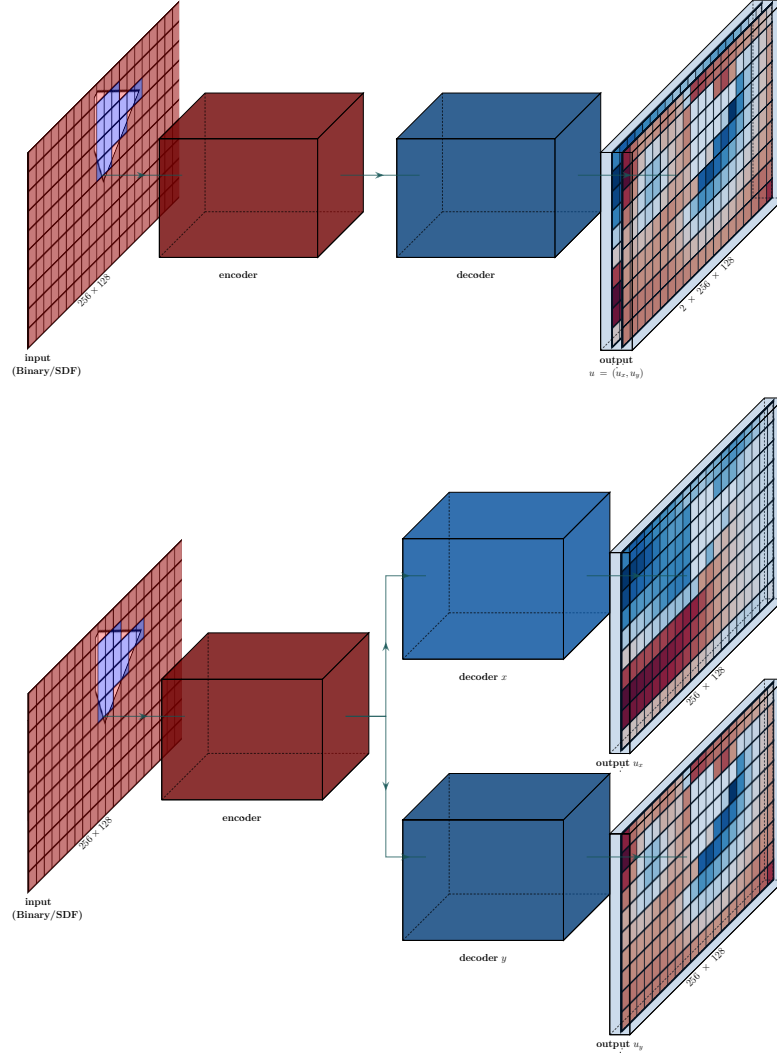


FIG. 3.2. Abstract graphical representation of the network architectures with one (top) or two (bottom) decoder paths. Visualization using [1].

3.2. One and two decoder paths. As in the work [16], we will also compare using one and two decoder paths in our neural networks. As pointed out in Section 3.1, both networks have a bottleneck type structure, where the first part corresponds to the encoder, which learns the geometrical features of the input image, and the second part corresponds to the decoder, which predicts the flow images in x and y direction based on the geometrical features from the encoder. Based on this structure of the network architecture, we investigate in Section 7 the case of training two separate decoder paths for the velocities in x and y direction. This clearly increases the number of parameters of the neural network because the decoder part is doubled in size; see Figure 3.2. On the other hand, this may help to better predict the two velocity components.

The total number of parameters of the bottleneck CNN and the U-Net with one or two decoder paths are listed in Table 6.1.

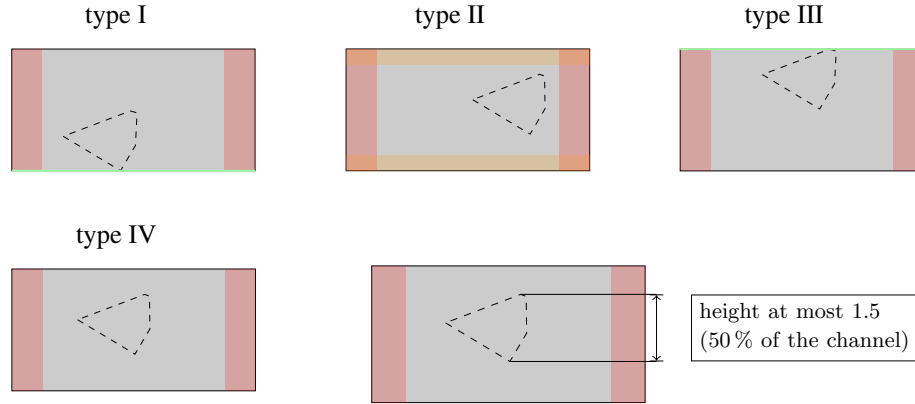


FIG. 4.1. Graphical representation of all geometric configurations considered in this paper. In all cases, a minimum distance of 1.5 to the inlet (left) and outlet (right) is assumed and the maximum height of the obstacle is 1.5, i.e., 50 % of the channel. Type I geometries touch the lower part of the boundary, type II geometries have a minimum distance of 0.75 to the lower and upper parts of the boundary, type III geometries touch the upper part of the boundary, and type IV have no restriction with respect to the vertical position of the obstacle.

4. Generation of training data. In order to build a surrogate machine learning model for the high-fidelity CFD simulations, we first have to generate a large set of flow data. To ensure validity of the model without imposing additional (physical) knowledge, this training data has to sufficiently cover the possible input space, i.e., the space of possible geometries; cf., e.g., [7]. In order to create such an extensive database, we have set up an automatized software pipeline, which enables the simulation of a large amount of different geometrical configurations, without the need for additional user interaction. This pipeline consists of the following steps, which will be discussed in the further subsections:

- Definition of an obstacle geometry; see Section 4.1
- Generation of a corresponding hexahedral simulation mesh; see Section 4.2
- CFD simulation using the finite volume solver OpenFOAM; see Section 4.3
- Data interpolation to a fixed pixel grid; see Section 4.4

We will further describe the specific data sets used in our experiments in the respective Sections 7 to 9.

4.1. Definition of the geometry. As previously described in Section 2, we will consider a two-dimensional channel $\Omega := [0, 6] \times [0, 3]$ and then remove a polygonal star-shaped obstacle P , such that the computational domain is $\Omega_P = \Omega \setminus P$. Since the channel Ω is fixed, the only freedom in the geometry is the definition of the polygonal obstacle. This is performed by a random placement of the corners of the polygon under the following restrictions.

In general, we assume that the obstacle has a minimum distance of 0.75 from the inlet and from the outlet, i.e., from the left and the right boundary of the channel. Furthermore, the vertical extension of the obstacle should not be larger than 1.5, that is 50 % of the height of the channel; cf. Figure 4.1 (bottom right). We first only consider obstacles which touch the lower boundary (type I), obstacles which do not touch any boundary and have a minimum distance of 0.75 to the lower and upper boundary (type II), and obstacles which touch the upper boundary (type III). Later, in Section 9, we will also consider type IV geometries, which have no restriction regarding the vertical location; see Figure 4.1.

In addition to the types I, II, and III, we also categorize the polygonal obstacles with respect to the number of edges. In general, we will only consider obstacles with less or equal

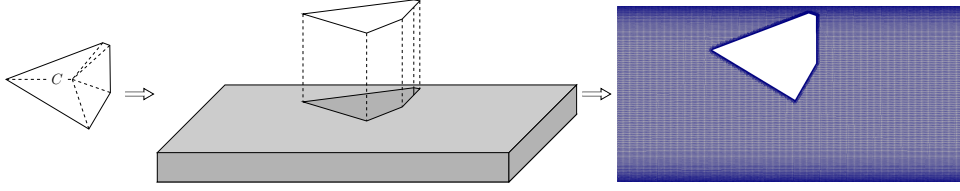


FIG. 4.2. Mesh generation using *snappyHexMesh*: The obstacle geometry is defined in *STL* format and placed within the channel. Then, the mesh is generated, such that it is additionally refined near $\partial\Omega_{\text{wall}}$.

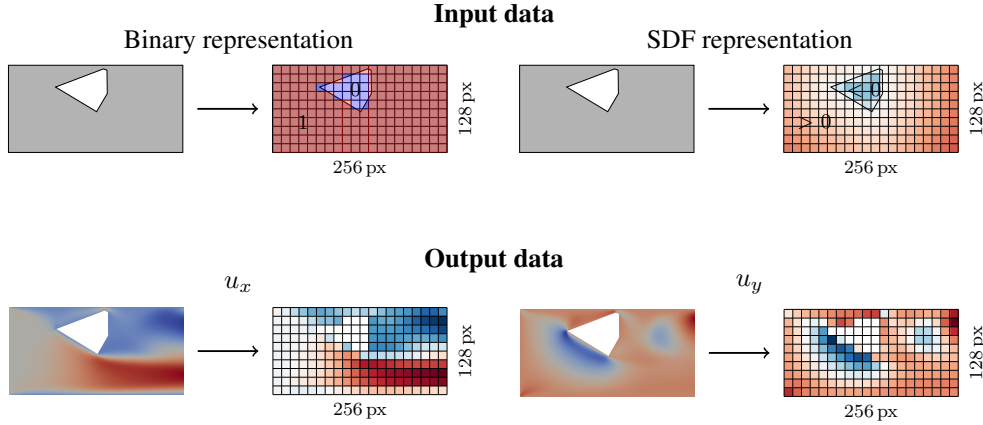


FIG. 4.3. The structured input and output data for the CNNs is obtained by evaluation in the centers of the pixels of a 256×128 pixel grid.

than 20 edges; only in Figure 7.4, we neglect this constraint and consider a circular obstacle.

Note that we generally do not constrain the angles of the polygons, such that we may obtain thin obstacles which can only be poorly resolved by our input image; cf. Section 4.4.

4.2. Mesh generation. The mesh generation procedure is sketched in Figure 4.2. In particular, we first describe the obstacle geometry, which has been previously generated along the description in Section 4.1, using the *STL* (Standard Triangle Language) format. Then, we use the mesh generator *blockMesh* to create an underlying hexagonal mesh of the channel Ω . Then, we cut out the obstacle and create a mesh, which is refined near $\partial\Omega_{\text{wall}}$, using the tool *snappyHexMesh*. Both tools, *blockMesh* and *snappyHexMesh*, are part of *OpenFOAM* 5.0 [14, 22] and will not be discussed here in detail.

4.3. OpenFOAM simulations. The CFD simulations of the steady Navier–Stokes equations with corresponding boundary conditions as described in Section 2 are performed using the *simpleFoam* solver in *OpenFOAM* 5.0 [14, 22]. In particular, the Navier–Stokes equations are first discretized using finite volumes [26, 37], and then, the resulting discrete nonlinear problem is solved using the SIMPLE (Semi-Implicit Method for Pressure Linked Equations) method [28]. Especially, the pressure equations are solved using an geometric algebraic multi grid (GAMG) solver and the remaining velocity equations are then solved using an symmetric Gauss–Seidel iteration. An exemplary solution is depicted in Figure 2.1. For more details, we refer to the *OpenFOAM* user guide [14].

4.4. Data interpolation. As already pointed out in Section 3, neural networks and, in particular, convolutional neural networks rely on input and output data with a fixed structure. Therefore, in order to train a neural network as a surrogate model for the previously described CFD simulations (Section 4.3), the input and output data has to be transformed to a fixed structure. In particular, we are using a CNN that requires an image structure for the data; cf. Section 3. Therefore, we will use two different image representations of the geometry, i.e., a signed distance function (SDF) representation and a binary representation; cf. [10, 16]. Both representations are 256×128 px images. Furthermore, we will interpolate the flow field onto a pixel grid of fixed 256×128 px size; see also Figure 4.3.

Signed distance function (SDF) input. The SDF input includes information about the obstacle as well as information about the minimum distance of any given pixel to the boundary of the obstacle. In particular, the signed distance function for a given pixel p is given as the minimum positive distance of the center of the pixel c_p to the boundary of the obstacle. However, in case that the center of the pixel lies in the interior of the obstacle, it is the negative distance.

Binary input. The binary input contains a reduced amount of information compared to the SDF input. In particular, the binary input value in a given pixel p is defined as zero if its center c_p is located in the interior of the obstacle and one otherwise. Hence, it can be obtained from the SDF input by mapping any positive value to one and any negative value to zero.

Output. The output data for a given configuration is obtained from the velocity field resulting from the corresponding OpenFOAM simulation; cf. Section 4.3. In particular, each component of the discrete solution $u = \begin{pmatrix} u_x \\ u_y \end{pmatrix}$ is interpolated onto a 256×128 pixel image by evaluation in the center c_p of each pixel p . Hence, we obtain two pixel images representing the velocity field for each configuration.

Note that, since we do not constrain the angles of the corners of the polygon, it is possible that the geometry cannot be resolved accurately by a 256×128 pixel grid.

5. Training. In the training phase, we optimize the parameters of the neural networks to minimize different loss functions. In particular, we consider combinations of a mean squared error (MSE) and a mean absolute error (MAE) with the mean relative error

$$(5.1) \quad \frac{1}{|D|} \sum_{P \in D} \frac{1}{|I_P|} \sum_{p \in I_P} \frac{\|u_p - \hat{u}_p\|_2}{\|u_p\|_2 + 10^{-4}},$$

where u_p is the velocity prediction vector in the pixel p and $\hat{u}_p$ is the reference velocity in p , i.e., the evaluation of the CFD solution in the center c_p of the pixel. Furthermore, D is the set of all considered obstacles P and I_P is the set of all pixels p that are not covered by the respective obstacle; hence, we neglect the error of the velocity prediction within the obstacle. We add 10^{-4} as a regularization term in the denominator in case of very small values in the reference velocity, i.e., $\|u_p\|_2 \approx 0$. In particular, we consider combinations with (5.1) because we will later use this as the error measure on the validation and test data in order to investigate the performance of the surrogate model; cf. (7.1). Note that this error measure alone was not sufficient as the loss function to train the neural networks, and a reasonable reduction of the loss during the optimization was only possible when adding either the MSE or the MAE.

TABLE 6.1

Training cost for the bottleneck CNN and the U-Net using one or two decoder paths. The times have been obtained from computations on a Nvidia GeForce RTX 2080Ti GPU; Section 6.

# decoders	Bottleneck CNN		U-Net	
	1	2	1	2
parameters	≈ 47 m	≈ 85 m	≈ 34 m	≈ 53.5 m
time/epoch	180 s	245 s	195 s	270 s

In total, we consider the four loss functions

$$\begin{aligned}
 \text{MSE} &= \frac{1}{|D|} \sum_{P \in D} \frac{1}{|I_P|} \sum_{p \in I_P} (\|u_p - \hat{u}_p\|_2^2), \\
 \text{MSE}^+ &= \frac{1}{|D|} \sum_{P \in D} \frac{1}{|I_P|} \sum_{p \in I_P} \left(\|u_p - \hat{u}_p\|_2^2 + \frac{\|u_p - \hat{u}_p\|_2}{\|u_p\|_2 + 10^{-4}} \right), \\
 \text{MAE} &= \frac{1}{|D|} \sum_{P \in D} \frac{1}{|I_P|} \sum_{p \in I_P} (\|u_p - \hat{u}_p\|_1), \text{ and} \\
 \text{MAE}^+ &= \frac{1}{|D|} \sum_{P \in D} \frac{1}{|I_P|} \sum_{p \in I_P} \left(\|u_p - \hat{u}_p\|_1 + \frac{\|u_p - \hat{u}_p\|_2}{\|u_p\|_2 + 10^{-4}} \right).
 \end{aligned}
 \tag{5.2}$$

Furthermore, in Section 7.2, we will discuss results for weighted variants of the MSE and MAE loss functions in order to improve the errors in the vicinity of the obstacles.

In order to optimize the parameters of the CNNs, we apply a stochastic gradient descent (SGD) up to a maximum number of 300 epochs; in case of stagnation in the reduction of loss for at least 50 epochs, we reduce the learning rate by 20 %. Moreover, we use a batch size of 64 and an adaptive scaling of the learning rate using the Adam (Adaptive moments) [23] algorithm with initial learning rate $\lambda = 0.001$. Furthermore, we observed that the training is improved if, in case of SDF input data, Z-normalization and, in case of binary input data, batch normalization is used; cf. [20].

6. Implementation and efficiency of the neural networks. Our implementation of the neural networks and the training algorithms uses Keras 2.2.4 [6] with Tensorflow 1.12 [2] backend.

On an AMD Threadripper 2950X (8×3.8 GHz) CPU with 32 GB RAM, the average time for a serial computation of one configuration, including the mesh generation and the CFD simulation, took in the order of $O(10)$ s. In comparison, the evaluation of our neural networks took in the order of $O(0.1)$ s on the same CPU. Using a Nvidia GeForce RTX 2080Ti GPU, the evaluation of the neural networks was again accelerated by a factor of approximately 20. Therefore, we can confirm that our surrogate convolutional neural networks are significantly more efficient in the online phase.

However, the training phase of the neural networks is quite expensive, due to the very large number of parameters in the neural network. As listed in Table 6.1, one single epoch (for 90 000 configurations of training data and 10 000 configurations of validation data) during the training process took approximately 180–270 s on the Nvidia GPU depending on the architecture of the network; the training cost on a CPU would be significantly higher. This highlights the high cost of the offline training phase.

7. Results on type I and II geometries. As a first step, we only consider geometries of type I and II; cf. Section 4.1 and Figure 4.1. In particular, using our software pipeline described

TABLE 7.1

*Comparison of the performance of the bottleneck CNN and the U-Net based on the error (7.1): variation of the input type, the number of decoder paths, and the loss function; cf. (5.2). The best errors for a given CNN architecture and input type are marked in **bold face**. Taken fom [10].*

input	# dec.	loss	Bottleneck CNN			U-Net		
			total	type I	type II	total	type I	type II
SDF	1	MSE	61.16 %	110.46 %	11.86 %	17.04 %	29.42 %	4.66 %
		MSE ⁺	3.97 %	3.31 %	4.63 %	2.67 %	2.11 %	3.23 %
		MAE	25.19 %	41.52 %	8.86 %	9.10 %	13.89 %	4.32 %
		MAE ⁺	4.45 %	3.84 %	5.05 %	2.48 %	1.87 %	3.10 %
	2	MSE	49.82 %	89.12 %	10.51 %	13.01 %	21.59 %	4.42 %
		MSE ⁺	3.85 %	3.05 %	4.64 %	2.43 %	1.78 %	3.23 %
		MAE	45.23 %	81.38 %	9.08 %	5.47 %	7.06 %	3.89 %
		MAE ⁺	4.33 %	3.74 %	4.91 %	2.57 %	1.98 %	3.17 %
Binary	1	MSE	49.78 %	88.28 %	11.28 %	27.15 %	49.15 %	5.15 %
		MSE ⁺	10.12 %	11.44 %	8.80 %	5.49 %	6.25 %	4.74 %
		MAE	39.16 %	64.77 %	13.54 %	15.69 %	26.36 %	5.02 %
		MAE ⁺	10.61 %	12.34 %	8.87 %	4.48 %	5.05 %	3.90 %
	2	MSE	51.34 %	91.20 %	11.48 %	24.00 %	43.14 %	4.85 %
		MSE ⁺	10.03 %	11.37 %	8.69 %	5.56 %	6.79 %	4.33 %
		MAE	37.16 %	62.01 %	12.32 %	21.54 %	38.12 %	4.96 %
		MAE ⁺	9.53 %	10.91 %	8.15 %	6.04 %	7.88 %	4.20 %

in Section 4, we first generate a data set consisting of 100 000 geometry configurations, where 50 000 configurations correspond to type I obstacles and the remaining 50 000 to type II obstacles. Moreover, we restrict ourselves to polygonal obstacles with 3, 4, 5, 6, and 12 edges; 10 000 each for type I and type II. Each obstacle is randomly generated under the conditions described in Section 4.1. In order to train our neural networks, we perform a random split into 90 000 training data and 10 000 validation data.

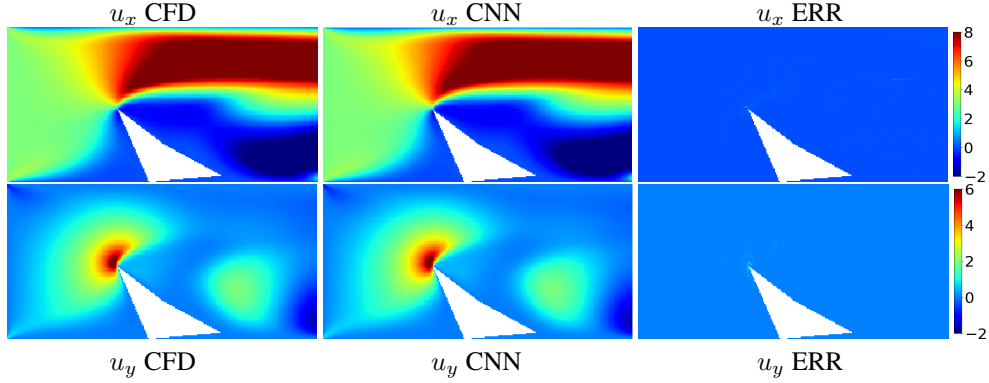
As an error measure for our surrogate models, we use the mean relative error (5.1) evaluated on the validation data,

$$(7.1) \quad \frac{1}{|V|} \sum_{P \in V} \frac{1}{|I_P|} \sum_{p \in I_P} \frac{\|u_p - \hat{u}_p\|_2}{\|u_p\|_2 + 10^{-4}},$$

where V is the set of all validation data. This error function is not equal to one of the loss functions, which are minimized for the 90 000 training data, but is part of two of the four considered loss functions (5.2); also compare for the discussion in Section 5.

7.1. Results on the validation data. First, in Table 7.1, we compare the bottleneck CNN and the U-Net described in Section 3.1 using SDF and binary input data, one or two decoder paths, and the four different loss functions (5.2). We list the relative errors for type I and type II validation data separately and additionally specify the relative error over all validation data. We observe that the U-Net generally yields better results compared to the bottleneck CNN. However, the best total relative errors obtained for the bottleneck CNN and the U-Net architectures are both very good with 3.85 % and 2.43 %. We observe that the bottleneck CNN benefits significantly from the use of the SDF input, which contains additional information compared to the binary input. In particular, for the bottleneck CNN, the best total relative error improves from 9.53 % to 3.85 %. This is however only slightly better than the best results for

Type I:



Type II:

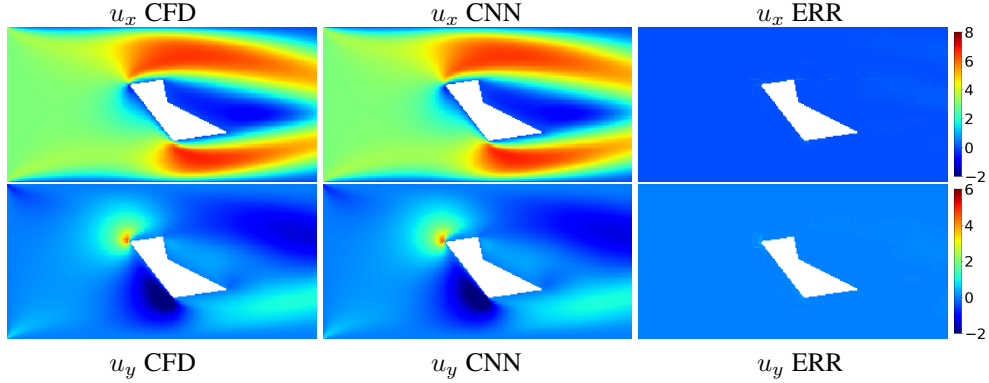


FIG. 7.1. Type I & II geometry: comparison of the CFD flow field (left) computed using OpenFOAM, the CNN prediction (middle), and the pointwise error (right) for x (top) and y (bottom) component. Both examples yield a low mean relative error (7.1) of approximately 2 %.

the U-Net for binary input data with 4.48 %. Hence, the U-Net seems to be able to compensate for the binary data due to its better overall performance. The U-Net produces good results for both input data types.

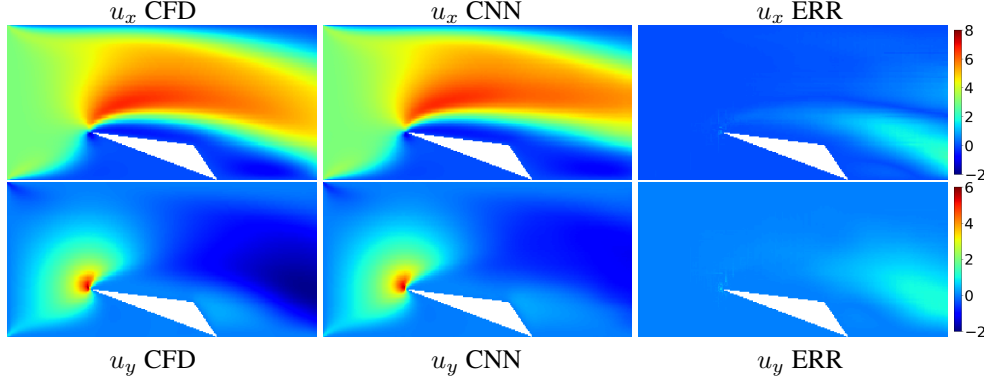
It is not clear whether MSE or MAE is the better choice as the basis for the loss function. However, we clearly observe that adding the mean relative error (5.1) to the loss always improves the results, for both network architectures. However, as already noted in Section 5, the networks could not be trained using just the mean relative error (5.1) as the loss function.

Furthermore, the results are not conclusive about whether a second decoder path should be used, or not. In particular, since the number of parameters and hence the training cost is significantly increased, it seems that the use of a single decoder is the more efficient choice.

Overall, many different configurations for the surrogate CNN yield very good results. However, we restrict ourselves to one specific network architecture and loss function for further experiments. In particular, following our findings, we will always consider the U-Net with one decoder path and MAE^+ loss.

Additionally, for this configuration and using SDF input, in Figure 7.1, we present exemplary results comparing the reference CFD solution and the CNN prediction. Whereas, in Figure 7.1, the results are in very good agreement qualitatively as well as quantitatively (errors of approximately 2 %), larger qualitative and quantitative differences can be observed

Type I:



Type II:

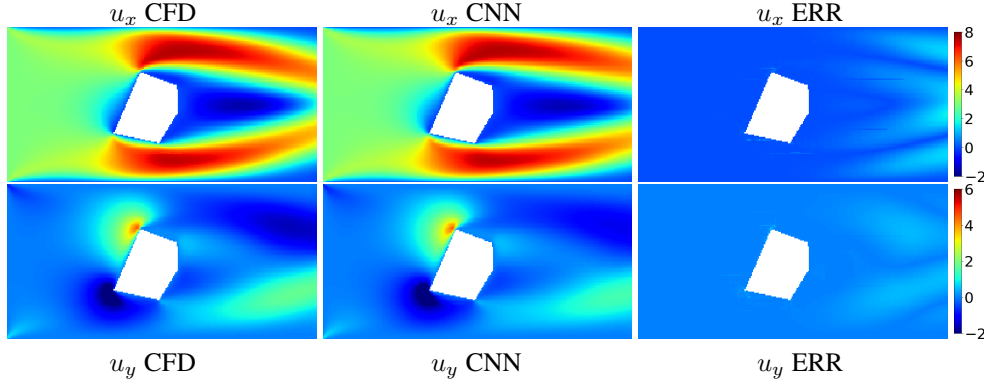


FIG. 7.2. Type I & II geometry: comparison of the CFD flow field (left) computed using OpenFOAM, the CNN prediction (middle), and the pointwise error (right) for x (top) and y (bottom) component. Both examples yield higher mean relative errors (7.1) of approximately 17 % (type I) and 15 % (type II).

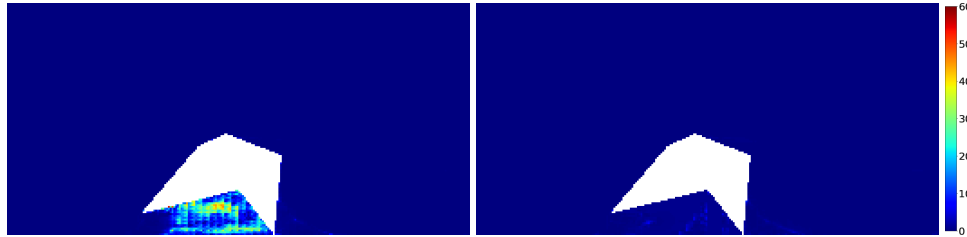


FIG. 7.3. Visualization of the pixel-wise relative error (7.7) for a type I geometry. Comparison for the unweighted MAE loss function (5.2) (left) and the weighted MAE loss function (7.3) with $\omega = 250$ (right).

in Figure 7.2 (errors of approximately 17 % and 15 %); see also [10] for additional results. To avoid these outliers and further reduce the errors will be subject of future research.

7.2. Spatial weighting of the loss function. In many applications, the velocities near the obstacle walls are of particular interest, e.g., for the computation of the wall shear stresses. In order to improve the accuracy of the prediction in the vicinity of the obstacles, we investigate

TABLE 7.2

Results on validation data for a higher weight of the loss corresponding to errors in the vicinity of the obstacle using a U-Net with one decoder path and SDF input. The weight parameter ω in the function (7.4) is varied between 1 and 500, and the global error (7.1) as well as split of the error into pixels with a maximum distance of 0.5 to the obstacle (error (7.5)) and remaining pixels (error (7.6)) are listed. The best errors among the different weight factors are marked in **bold face**.

loss	ω	error (7.1)			error (7.5)		error (7.6)	
		total	type I	type II	type I	type II	type I	type II
MSE_ω (7.2)	1	17.04 %	29.42 %	4.66 %	28.24 %	2.39 %	1.19 %	2.29 %
	50	9.41 %	13.25 %	5.56 %	9.95 %	1.78 %	3.32 %	3.79 %
	100	12.11 %	18.32 %	5.90 %	14.74 %	1.76 %	3.59 %	4.16 %
	250	10.95 %	14.21 %	7.68 %	8.19 %	1.69 %	6.04 %	6.01 %
	500	13.94 %	17.72 %	10.16 %	9.89 %	2.10 %	7.85 %	8.07 %
MAE_ω (7.3)	1	9.11 %	13.89 %	4.32 %	12.96 %	2.19 %	0.94 %	2.14 %
	50	4.04 %	4.55 %	3.53 %	3.51 %	1.28 %	1.05 %	2.27 %
	100	3.68 %	3.82 %	3.54 %	2.61 %	1.19 %	1.22 %	2.37 %
	250	3.63 %	3.64 %	3.62 %	2.33 %	1.18 %	1.32 %	2.45 %
	500	3.83 %	3.88 %	3.78 %	2.46 %	1.19 %	1.43 %	2.60 %

weighted variants of the MSE and MAE loss functions,

$$(7.2) \quad MSE_\omega = \frac{1}{|D|} \sum_{P \in D} \frac{1}{|I_P|} \sum_{p \in I_P} W_\omega(p) (\|u_p - \hat{u}_p\|_2^2) \text{ and}$$

$$(7.3) \quad MAE_\omega = \frac{1}{|D|} \sum_{P \in D} \frac{1}{|I_P|} \sum_{p \in I_P} W_\omega(p) (\|u_p - \hat{u}_p\|_1)$$

with the weight function

$$(7.4) \quad W_\omega(p) = \max \left\{ 1, \omega e^{-2 \ln(\omega) \|c_p - P\|} \right\}$$

and the weight parameter $\omega > 1$. This weight function is chosen to be high at the boundary of the obstacle, i.e., $W_\omega(p) = \omega$ for $\|c_p - P\| = 0$. Then, the function decays exponentially and reaches a value of 1 in a distance of 0.5 to the obstacle, i.e., $W_\omega(p) = 1$ for $\|c_p - P\| \geq 0.5$. Moreover, if $\omega = 1$, $W(p) \equiv 1$ and the weighted loss functions are equal to the unweighted loss functions.

In order to investigate the effect of the weight function $W(p)$ on the error in the vicinity of the obstacle, we investigate a split of the error function (7.1) into

$$(7.5) \quad \frac{1}{|V|} \sum_{P \in V} \frac{1}{|I_P|} \sum_{\substack{p \in I_P \\ \|c_p - I_P\| \leq 0.5}} \frac{\|u_p - \hat{u}_p\|_2}{\|u_p\|_2 + 10^{-4}},$$

which corresponds to the error in the pixels near the obstacle, and

$$(7.6) \quad \frac{1}{|V|} \sum_{P \in V} \frac{1}{|I_P|} \sum_{\substack{p \in I_P \\ \|c_p - I_P\| > 0.5}} \frac{\|u_p - \hat{u}_p\|_2}{\|u_p\|_2 + 10^{-4}},$$

which corresponds to the remainder of the channel.

TABLE 7.3

Results for the generalization properties of the U-Net with one decoder path, and MAE⁺ loss function. Error (7.1) for polygonal obstacles with higher numbers of edges compared to the training and validation data: 1 000 polygons (500 type I and 500 type II) for each number of edges; cf. [10].

# polygon edges	SDF input			Binary input		
	total	type I	type II	total	type I	type II
7	2.71 %	1.89 %	3.53 %	4.39 %	4.61 %	4.16 %
8	2.82 %	1.98 %	3.65 %	4.67 %	4.89 %	4.44 %
10	3.21 %	2.32 %	4.10 %	5.23 %	5.51 %	4.94 %
15	4.01 %	3.16 %	4.86 %	7.76 %	7.85 %	6.66 %
20	5.08 %	4.22 %	5.93 %	9.70 %	10.43 %	8.97 %

We present our results for varying values of ω in Table 7.2. As can be observed, the errors (7.5) and (7.6) are comparable for type II geometries, whereas the error near the obstacle is significantly higher for type I geometries; see Figure 7.3 (left) for an example of a type I obstacle with high relative errors

$$(7.7) \quad \frac{\|u_p - \hat{u}_p\|_2}{\|u_p\|_2 + 10^{-4}}.$$

in pixels near the obstacle for the unweighted MAE loss function (5.2). Hence, it is particularly important to improve the error near the obstacle for type I geometries.

The results in Table 7.2 show that, by increasing the weight parameter ω , we can reduce the error near the obstacle (7.5) while only slightly increasing the error in the remainder of the channel (7.6). The error reduction can also be seen in the example in Figure 7.3, where the pixel-wise relative error (7.7) near the obstacle is significantly reduced when using the weighted MAE loss function (7.3) with $\omega = 250$. Moreover, by choosing an appropriate value for ω , we can even improve the global error (7.1) averaged over all geometries compared to the unweighted loss functions. This shows that the distribution of the error can be controlled in a reasonable way by using a weighted loss function.

In the remainder of this paper, we will focus on using the MAE⁺ loss function. However, it may be helpful in certain applications to also consider a weighted variant of this loss function.

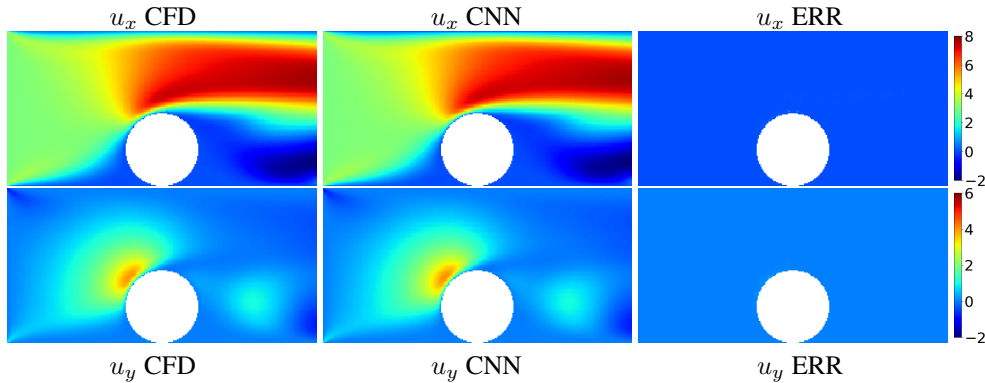
7.3. Generalization to other type I and II geometries. In order to study the generalization properties of our U-Net-based surrogate models, we consider additional geometries that have not been part of the initial data set. In particular, we consider polygonal obstacles with 7, 8, 10, 15, and 20 edges using the U-Net with one decoder path and MAE⁺ loss function. Furthermore, we consider both SDF and binary input.

As can be observed in Table 7.3, the test results on geometries that were not part of the initial data set are very good with a maximum total error of less than 10 %. Again, the results are better using SDF input. Of course, the flow fields may become more complex for higher numbers of edges, and the corresponding network predictions become slightly worse.

In addition to that, we depict the results for two configurations with circular obstacles. The first example is of type I and the second example is a type II circular obstacle inspired by the benchmark problem introduced in [35]; see Figure 7.4. For these examples, we obtain good generalization results with mean relative errors of approximately 1 % and 3 %.

Finally, in Figure 7.5, we provide three examples of geometries which are clearly outside the validity range of our CNN model: a type III obstacle, two type II obstacles, and one large type II obstacle. For all corresponding model predictions, a degenerated flow field is

Type I:



Type II:

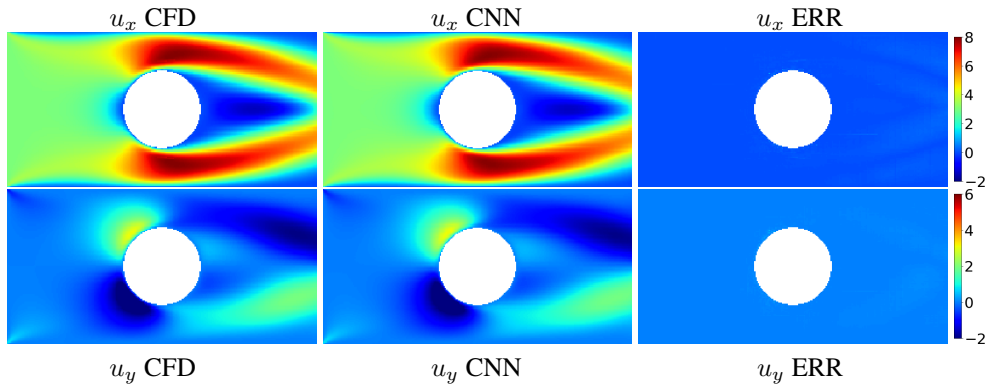


FIG. 7.4. Comparison of the CFD flow field (top) computed using OpenFOAM, the CNN prediction (middle), and the pointwise error (bottom) for x (left) and y (right) component. Results for a geometry with circular obstacles of type I & II, which have not been part of the training or validation data. Nonetheless, the mean relative errors (7.1) are only approximately 1 % (type I) and 3 % (type II).

TABLE 8.1

Application to type III data of the U-Net with one decoder path, and MAE^+ loss function trained on type I and II data. The mean relative error (7.1) is given.

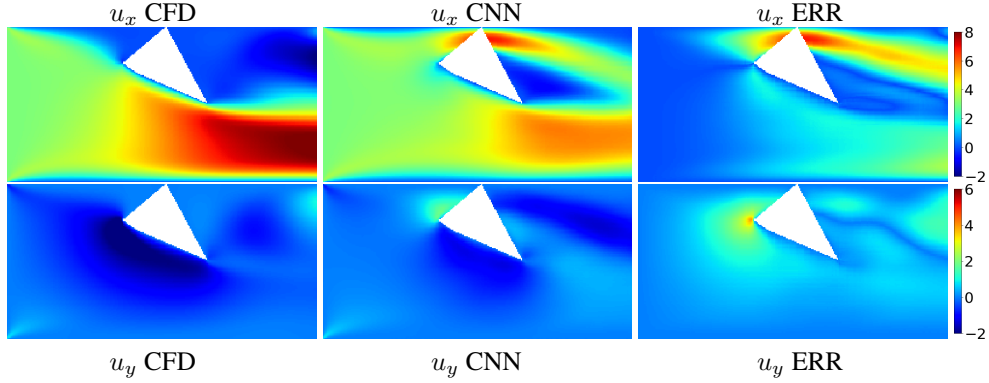
	SDF input	Binary input
type III	22 985.89 %	4 134.69 %

visible, and the prediction error clearly deteriorates. In particular, we can observe that model prediction would rather fit to the case of a type II obstacle.

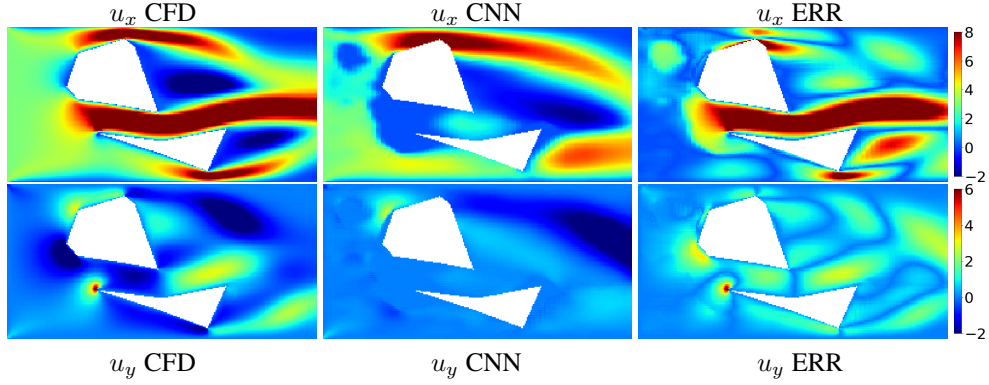
In order to further extend the validity range of our model and to overcome these issues, we will now discuss the application of transfer learning techniques. In particular, we focus on transfer learning for type III geometries.

8. Transfer learning for type III geometries. In order to investigate the generalization of our model to type III geometries, we generate a total of 5 000 additional polygonal obstacles of type III with 3, 4, 5, 6, and 12 edges; cf. Section 4.1 and Figure 4.1 for the description of type III obstacles.

Type III:



2 × Type II:



Large Type II:

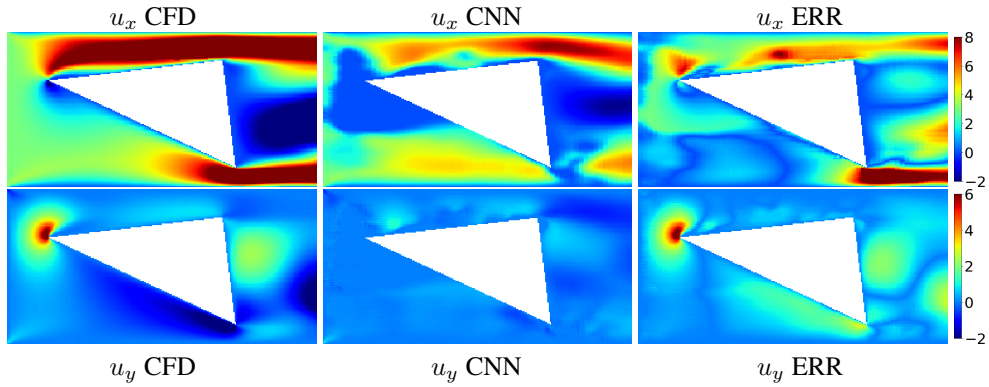


FIG. 7.5. Comparison of the CFD flow field (top) computed using OpenFOAM, the CNN prediction (middle), and the pointwise error (bottom) for x (left) and y (right) component. Results for three types of geometries which have not been part of the training or validation data: a type III obstacle, two type II obstacles, and one large type II obstacle. The network that was trained on type I and type II geometries is not able to generalize to these geometries, which are clearly outside the validity range of the model. Mean relative errors (7.1) of approximately 15 777 % (type III obstacle), 140 % (two type II obstacles), and 67 % (large type II obstacle) are obtained.

TABLE 8.2

Results of the error (7.1) for different learning approaches for type III data: training with random initial guess for type III data (approach 1), using a pre-trained neural network (on type I and II geometries) as the initial guess in training for type III data (approach 2), and using a pre-trained neural network (on type I and II geometries) as the initial guess in training for a combined data set with type I, II, and III data.

learning approach	# training epochs	type I & II		type III	
		SDF input	Binary input	SDF input	Binary input
1	100	-	-	98.02 %	111.75 %
2	100	208.02 %	105.43 %	7.18 %	11.81 %
3	3	3.33 %	7.06 %	4.94 %	11.28 %

As can be observed in Table 8.1, the prediction performance on 2 500 randomly chosen validation data deteriorates independent of the input type; the errors 22 985.89 % and 4 134.69 % indicate that the neural network was essentially not able to predict the flow at all; see also the example in Figure 7.5. As previously described in Section 4, this is due to the fact that the training data does not sufficiently cover the possible input geometries of type III; see also [7].

Now, we consider three different strategies to train type III geometries:

1. We train our U-Net with random initial parameters only using 2 500 training and 2 500 validation data of type III.
2. We use the trained U-Net from Section 7 as the initial guess for training with only the 2 500 training and 2 500 validation data of type III.
3. We use the trained U-Net from Section 7 as the initial guess for training on a combined data set. In particular, we use the 90 000 training and 10 000 validation data for type I and II geometries from Section 7 and add the 2 500 training and 2 500 validation data of the type III geometries.

The results for the three different learning approaches are listed in Table 8.2. Using the first approach, we are not able to obtain any good results after 100 epochs of training on type III geometries; the error is in the order of 100 % for both SDF and binary input data. Presumably, this is due to the too small amount of training data. The second approach yields good results for type III geometries, which can be attributed to the good initial guess for the parameters of the neural network. However, since no type I or II geometries were part of the training data for the last 100 epochs, the corresponding prediction accuracy deteriorated. Our third approach is able to maintain the good prediction properties with respect to type I and II data while providing even better results for type III geometries compared to the second approach after only 3 epochs of additional training. This is remarkable since the same amount of type III as in the first two approaches has been used.

9. Sequential learning strategy. Based on the very promising results of the third transfer learning approach for type III geometries in Section 8, we now propose a sequential learning strategy. The main idea is to start with a rather small training data set and enlarge it iteratively until a certain threshold for the validation error is reached. Therefore, we are able to reduce the amount of necessary data significantly.

First, we apply a sequential learning strategy for type I–III geometries. In particular, we first generate a fixed set of 10 000 validation data of type I, II, and III with polygonal obstacles with 3, 4, 5, 6, and 12 edges. We start with 2 000 random configurations of type I–III geometries as training data and random parameters of the neural network. Then, we iteratively add 2 000 additional random configurations and train the U-net using the third transfer learning

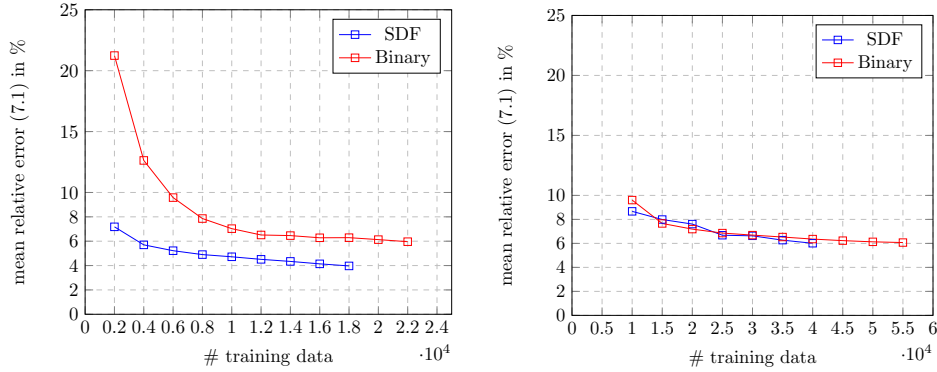


FIG. 9.1. Mean relative error (7.1) for our sequential learning strategy as described in Section 9: results for type I–III data (left) and for type IV data (right).

strategy from Section 8; in particular, we always use the neural network from the previous iteration as the initial guess. This iterative process is repeated until the error on the fixed 10 000 validation data reaches as prescribed threshold. Here, we stop the sequential learning when the validation error is below 4 % or 6 % for the SDF or the binary input, respectively. As can be observed from Figure 9.1 (left), 18 000 configurations are sufficient to obtain a validation error of approximately 4 % for SDF input and 22 000 configurations with binary input to obtain an error of approximately 6 %. Hence, we are able to obtain better results compared to Sections 7 and 8 using approximately 20 % of the data; note that the data set from Section 7 is actually simpler since it only contains type I and II geometries, whereas the data from Section 8 also contains type I–III geometries but does not have exactly the same distribution of the different types of geometries.

Finally, we apply the same sequential learning strategy to type IV data, i.e., data with no restriction of the vertical position; cf. Section 4.1. This data set is even more general than type I–III. In particular, it includes geometries that are close to the lower and upper wall, resulting in relatively high velocities close to these walls. Therefore, we start with a larger initial training data set of 10 000 and enlarge it by 5 000 in each iteration. We observe that a significantly higher amount of data is necessary to obtain an error of approximately 6 % compared to type I–III geometries. In particular, 40 000 and 55 000 configurations are finally employed for SDF (6.01 % validation error) and binary input (6.07 % validation error), respectively; cf. Figure 9.1 (right). However, this is still only approximately 50 % of the data used in Sections 7 and 8. One further observation is that the errors for SDF and binary input do not differ as much as for the previous data sets.

Overall, using our sequential learning strategy, we are able to obtain very good prediction results with a relatively low amount of data.

10. Conclusion. In this paper, we have extensively investigated our approach of constructing a surrogate model for high-fidelity CFD simulations using convolutional neural networks; see also [10, 16]. In particular, in order to facilitate the prediction of flow fields for varying geometries, we employed convolutional neural networks mapping from a pixel image of the geometry to pixel images of velocity components in the computational domain. In this regard, our approach differs from the majority of the existing approaches, where a single neural network is used to discretize the solution of one single boundary value problem with a fixed geometry, whereas our approach is able to predict flow fields for varying geometries.

In order to investigate our approach, we have set up a software pipeline enabling the generation of a large set of geometry and flow data. In a comparison of two convolutional neural network architectures using one or two decoders, different loss functions, and different input data, we have found that, in particular, the U-Net architecture, which has originally been introduced for biomedical image segmentation, is very robust and yields good predictions. Furthermore, we have obtained good generalization properties when applying the trained neural network to unseen geometries of the same type. Furthermore, we have presented an approach to reduce the error near the obstacles by introducing higher weights for the corresponding loss terms.

In order to transfer the model to new types of geometries, we have investigated an efficient transfer learning approach. Based on this, we have also described a sequential learning approach, where we iteratively enlarge the data set until an acceptable validation error is obtained. Using this approach, we were able to significantly reduce the amount of necessary training data.

On one exemplary machine, the evaluation of the convolutional neural networks is in the order of 100 times faster compared to generating a computational mesh and performing a CFD simulation using OpenFOAM.

REFERENCES

- [1] *PlotNeuralNet GitHub repository*. <https://github.com/HarisIqbal88/PlotNeuralNet>. Accessed: 2020-08-14.
- [2] M. ABADI, A. AGARWAL, P. BARHAM, E. BREVDO, Z. CHEN, C. CITRO, G. S. CORRADO, A. DAVIS, J. DEAN, M. DEVIN, S. GHEMAWAT, I. GOODFELLOW, A. HARP, G. IRVING, M. ISARD, Y. JIA, R. JOZEFOWICZ, L. KAISER, M. KUDLUR, J. LEVENBERG, D. MANÉ, R. MONGA, S. MOORE, D. MURRAY, C. OLAH, M. SCHUSTER, J. SHLENS, B. STEINER, I. SUTSKEVER, K. TALWAR, P. TUCKER, V. VANHOUCHE, V. VASUDEVAN, F. VIÉGAS, O. VINYALS, P. WARDEN, M. WATTENBERG, M. WICKE, Y. YU, AND X. ZHENG, *TensorFlow: Large-scale machine learning on heterogeneous systems*. e-print arXiv:1603.04467, March 2016. <https://arxiv.org/abs/1603.04467>
Software available from <https://tensorflow.org>.
- [3] N. BAKER, F. ALEXANDER, T. BREMER, A. HAGBERG, Y. KEVREKIDIS, H. NAJM, M. PARASHAR, A. PATRA, J. SETHIAN, S. WILD, AND K. WILLCOX, *Brochure on basic research needs for scientific machine learning: Core technologies for artificial intelligence*, U.S. Dept. of Energy (2018), Art. 1484362 (4 pages). <http://dx.doi.org/10.2172/1484362>.
- [4] A. BECK, D. FLAD, AND C.-D. MUNZ, *Deep neural networks for data-driven LES closure models*, J. Comput. Phys., 398 (2019), Art. 108910 (23 pages).
- [5] F. CHOLLET, *Deep Learning with Python*, 1st ed., Manning Publications, Shelter Island, 2017.
- [6] F. CHOLLET ET AL., *Keras*, 2015. <https://keras.io>
- [7] P. COURRIEU, *Three algorithms for estimating the domain of validity of feedforward neural networks*, Neural Netw., 7 (1994), pp. 169–174.
- [8] N. DAL SANTO, S. DEPARIS, AND L. PEGOLOTTI, *Data driven approximation of parametrized PDEs by reduced basis and neural networks*, J. Comput. Phys., 416 (2020), Art. 109550 (21 pages).
- [9] W. E AND B. YU, *The Deep Ritz method: A deep learning-based numerical algorithm for solving variational problems*, Commun. Math. Statist., 6 (2018), pp. 1–12.
- [10] M. EICHINGER, A. HEINLEIN, AND A. KLAWONN, *Stationary flow predictions using convolutional neural networks*, in Numerical Mathematics and Advanced Applications ENUMATH 2019, F. J. Vermolen and C. Vuik, eds., Springer, Cham, 2021, pp. 541–549.
- [11] S. FRESCA, A. MANZONI, L. DEDÉ, AND A. QUARTERONI, *Deep learning-based reduced order models in cardiac electrophysiology*, e-print arXiv:2006.03040, June 2020. <https://arxiv.org/abs/2006.03040>
- [12] X. GLOROT, A. BORDES, AND Y. BENGIO, *Deep sparse rectifier neural networks*, in Proceedings of Machine Learning Research, G. Gordon, D. Dunson, and M. Dudík, eds., vol. 15 of JMLR Workshop and Conference Proceedings, 2011, pp. 315–323.
- [13] I. GOODFELLOW, Y. BENGIO, AND A. COURVILLE, *Deep Learning*, MIT Press, Cambridge, 2016.
- [14] C. J. GREENSHIELDS, *OpenFOAM user guide*, v5. 0, OpenFOAM foundation, 2017.
- [15] V. GRIMM, A. HEINLEIN, A. KLAWONN, M. LANSER, AND J. WEBER, *Estimating the time-dependent contact rate of SIR and SEIR models in mathematical epidemiology using physics-informed neural*

- networks, *Electron. Trans. Numer. Anal.*, 56 (2022), pp. 1–27.
<https://etna.ricam.oeaw.ac.at/vol.56.2022/pp1-27.dir/pp1-27.pdf>
- [16] X. GUO, W. LI, AND F. IORIO, *Convolutional neural networks for steady flow approximation*, in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining KDD'16*, ACM, New York, 2016, pp. 481–490.
 - [17] D. HARTMANN, C. LESSIG, N. MARGENBERG, AND T. RICHTER, *A neural network multigrid solver for the Navier-Stokes equations*, e-print arXiv:2008.11520, August 2020.
<https://arxiv.org/abs/2008.11520>
 - [18] A. HEINLEIN, A. KLAOWONN, M. LANSER, AND J. WEBER, *Machine learning in adaptive domain decomposition methods—Predicting the geometric location of constraints*, *SIAM J. Sci. Comput.*, 41 (2019), pp. A3887–A3912.
 - [19] M. W. HESS, A. QUAINI, AND G. ROZZA, *A comparison of reduced-order modeling approaches for pdes with bifurcating solutions*, *Electron. Trans. Numer. Anal.*, 56 (2022), pp. 52–65.
<https://etna.ricam.oeaw.ac.at/vol.56.2022/pp52-65.dir/pp52-65.pdf>
 - [20] S. IOFFE AND C. SZEGEDY, *Batch normalization: Accelerating deep network training by reducing internal covariate shift*, e-print arXiv:1502.03167, February 2015. <https://arxiv.org/abs/1502.03167>
 - [21] K. JARRETT, K. KAVUKCUOGLU, M. RANZATO, AND Y. LECUN, *What is the best multi-stage architecture for object recognition?*, in *Proceedings of the 2009 IEEE 12th International Conference on Computer Vision*, IEEE Conference Proceedings, Los Alamitos, 2010, pp. 2146–2153.
 - [22] H. JASAK, *OpenFOAM: Open source CFD in research and industry*, *Int. J. Nav. Archit.*, 1 (2009), pp. 89–94.
 - [23] D. P. KINGMA AND J. BA, *Adam: A method for stochastic optimization*, e-print arXiv:1412.6980, December 2014. <https://arxiv.org/abs/1412.6980>
 - [24] I. E. LAGARIS, A. LIKAS, AND D. I. FOTIADIS, *Artificial neural networks for solving ordinary and partial differential equations*, *IEEE Trans. Neural Netw.*, 9 (1998), pp. 987–1000.
 - [25] Y. LECUN, *Generalization and network design strategies*, *Connectionism Perspect.*, 19 (1989), pp. 143–155.
 - [26] F. MOUKALLED, L. MANGANI, AND M. DARWISH, *The Finite Volume Method in Computational Fluid Dynamics*, Springer, Cham, 2016.
 - [27] V. NAIR AND G. E. HINTON, *Rectified linear units improve restricted Boltzmann machines*, in *Proceedings of the 27th International Conference on Machine Learning*, J. Fürnkranz and T. Joachims, eds., Omnipress, Madison, 2010, pp. 807–814.
 - [28] S. PATANKAR AND D. SPALDING, *A calculation procedure for heat, mass and momentum transfer in three dimensional parabolic flows*, *Int. J. Heat Mass Transf.*, 15 (1972), pp. 1787–1806.
 - [29] K. PEARSON, *On lines and planes of closest fit to systems of points in space*, *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2 (1901), pp. 559–572.
 - [30] A. QUARTERONI, A. MANZONI, AND F. NEGRI, *Reduced Basis Methods for Partial Differential Equations: An Introduction*, Springer, Cham, 2016.
 - [31] M. RAISSI, P. PERDIKARIS, AND G. E. KARNIADAKIS, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, *J. Comput. Phys.*, 378 (2019), pp. 686–707.
 - [32] M. RATHINAM AND L. R. PETZOLD, *A new look at proper orthogonal decomposition*, *SIAM J. Numer. Anal.*, 41 (2003), pp. 1893–1925.
 - [33] D. RAY AND J. S. HESTHAVEN, *An artificial neural network as a troubled-cell indicator*, *J. Comput. Phys.*, 367 (2018), pp. 166–191.
 - [34] O. RONNEBERGER, P. FISCHER, AND T. BROX, *U-net: Convolutional networks for biomedical image segmentation*, in *Medical Image Computing and Computer-Assisted Intervention—MICCAI 2015*, N. Navab, J. Hornegger, W. Wells, A. Frangi, eds., *Lect. Notes Comput. Sci.*, vol. 9351, Springer, Cham, 2015, pp. 234–241.
 - [35] M. SCHÄFER, S. TUREK, F. DURST, E. KRAUSE, AND R. RANNACHER, *Benchmark computations of laminar flow around a cylinder*, in *Flow Simulations with High-Performance Computers II*, E. H. Hirschel, ed., *NONUFM Vol. 48*, Vieweg, Wiesbaden, 1996, pp. 547–566.
 - [36] W. H. SCHILDERS, H. A. VAN DER VORST, AND J. ROMMES, *Model Order Reduction: Theory, Research Aspects and Applications*, Springer, Berlin, 2008.
 - [37] P. WESSELING, *Principles of Computational Fluid Dynamics*, Springer, Berlin, 2001.